Memorisation in LLM-Based Pentesting: Strength or Weakness?

ARIFANSYAH WICAKSONO, Monash University, Australia
XIAONING DU, Monash University, Australia

LOREM IPSUM

Additional Key Words and Phrases: Do, Not, Use, This, Code, Put, the, Correct, Terms, for, Your, Paper

ACM Reference Format:
Arifansyah Wicaksono and Xiaoning Du. 2018. Memorisation in LLM-Based Pentesting: Strength or Weakness?. *ACM Trans. Graph.* 37, 4, Article 111 (August 2018), 5 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large Language Models (LLMs) have recently been in rapid development. For instance in penetration testing domain, LLMs are increasingly being adopted to assist human experts because they can recognise recurring attack patterns and respond flexibly to dynamic or uncertain testing environments [Happe and Cito 2025]. One of the LLM-based penetration testing tool PentestGPT [Deng et al. 2024] achieved 111% more completed sub-tasks than the baseline LLM. Subsequent studies have further extended LLM-based penetration testing towards autonomous operation [Dai et al. 2025; Gioacchini et al. 2024; Huang and Zhu 2023; Kong et al. 2025a; Xu et al. 2024].

A key challenge in LLM-based penetration testing is the tendency to lose contextual information as the testing process advances through multiple stages. To address this limitation, researchers have proposed practical frameworks designed to support more structured and effective LLM-based penetration testing. Some studies have also incorporated memory mechanisms and external knowledge bases to preserve context and improve the robustness of the generated actions [Challita and Parrend 2025]. However, stronger memorisation may also make it difficult to distinguish between genuine reasoning and the reproduction of previously observed attack patterns. To reduce the influence of memorisation, Zhu et al. [2025] evaluated LLM agents using zero-day vulnerabilities disclosed after the models knowledge cut-off dates, thereby minimising the risk of training data leakage caused by memorisation.

Memorisation in LLMs occurs when a model reproduces sequences from its training data verbatim [Biderman et al. 2023]. A common method for measuring and detecting potential memorisation is ground-truth matching, in which the output generated by an LLM is compared with known training data to identify exact matches [Carlini et al. 2019, 2021]. For instance, Carlini et al. [2021] used publicly available online data as ground truth and compared generated strings with search engine results. In contrast, Nasr et al. [2023] relied on a large corpus rather than search engine results for comparison. Beyond natural language generation, potential memorisation has also been identified in code generation [Chen et al. 2025] and in LLM-based automatic program repair [Kong et al. 2025b]. However, memorisation in LLM-based penetration testing remains underexplored.

Therefore, this research conducted by answering two research questions to explore the underexplored area:

(1) RQ1: How effective are LLM-based penetration testing tools on public and private vulnerable machines, and to what extent can memorisation be detected through ground truth matching with existing write-ups and solutions?
(2) RQ2: How do LLM-based penetration testing tools adapt when constraints, incomplete information, or memory-triggering scenarios are introduced, and does this reveal memorisation as a strength or a weakness?

The research identified the following key findings based on the research questions:

(1)

2 Background

2.1 LLM-based Penetration Testing

LLM-based penetration testing has emerged as an important research direction in cybersecurity, with recent studies exploring how large language models can support or automate different stages of offensive security assessment. Rather than functioning only as general-purpose chatbots, LLMs have increasingly been integrated into penetration testing workflows as assistants or autonomous agents capable of interpreting tool outputs, suggesting commands, planning attack paths, and generating reports. For example, Hilario et al. [2024] examined the use of ShellGPT to support penetration testers across multiple stages, from reconnaissance to exploitation, before using the generated outputs to produce a penetration testing report.

One of the earliest and most influential tools in this area is PentestGPT, introduced by Deng et al. [2024]. PentestGPT uses a three-module architecture consisting of a Reasoning module, a Generation module, and a Parsing module. The Reasoning module constructs and updates a penetration testing tree, the Generation module produces commands using chain-of-thought prompting, and the Parsing module improves token efficiency by summarising user intentions, tool outputs, HTTP information, and source code. PentestGPT demonstrated the potential of LLM-assisted penetration testing by solving 5 out of 10 active HackTheBox machines [HACKTHEBOX [n. d.]]. Building on this work, Isozaki et al. [2025] conducted an ablation study using PentestGPT to evaluate the effects of summary injection, structured generation, and Retrieval-Augmented

Authors' Contact Information: Arifansyah Wicaksono, awic0022@student.monash.edu, Monash University, Clayton, Victoria, Australia; Xiaoning Du, xiaoning.du@monash.edu, Monash University, Clayton, Victoria, Australia.

ACM 1557-7368/2018/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

Generation (RAG), with HackTricks used as an external knowledge base.

Subsequent studies have extended LLM-based penetration testing towards more autonomous and robust frameworks. RedTeamLLM was developed to address limitations such as context window constraints, the need for continuous improvement, lack of generality, and limited automation. It adopts a modular architecture consisting of several components, including a launcher, central RedTeamAgent, memory manager, task decomposition module, plan corrector, ReAct-based executor, and planner. The framework also integrates memory mechanisms to store execution summaries and embeddings for retrieval. RedTeamLLM was reported to outperform PentestGPT on three machines [Challita and Parrend 2025]. However, its evaluation was limited to five archived machines, whereas PentestGPT was evaluated on both archived and active machines, demonstrating its usefulness in assisting human testers in more dynamic environments.

Other studies have focused on increasing autonomy in LLM-based penetration testing. Happe and Cito [2023] connected LLMs to an SSH environment through a Python script to support automated privilege escalation and remote shell access. Their study divided the execution process into high-level planning and low-level command execution. The results showed that LLMs can generate feasible attack scenarios at a high level, although hallucination remains a concern, as some generated outputs were nonsensical or unreliable. Nevertheless, the model was still able to suggest reasonable actions in several cases.

Context loss has also become a recurring challenge in LLM-based penetration testing, especially when attacks involve multiple steps, large tool outputs, or lateral movement across systems. To address this, Xu et al. [2024] proposed AutoAttacker, a system designed to support cross-machine attacks and mitigate context loss. AutoAttacker consists of four main modules: a Summariser for past actions, a Planner for generating concrete actions, an Experience Manager that stores and retrieves successful actions using vector embeddings and RAG, and a Navigator that selects effective actions based on the planner and retrieved experience. Similarly, Dai et al. [2025] developed RefPentester, which combines self-reflection memory with an external knowledge base. RefPentester uses success logs as long-term memory and failure logs as short-term memory, while also retrieving external knowledge from a vector database through RAG. Although this memory-based design improved performance over the GPT-4o baseline, the study did not conduct an ablation analysis to determine the extent to which the memory mechanism itself contributed to the results.

Other frameworks have expanded the scope of LLM-based penetration testing beyond exploitation. Huang and Zhu [2023] proposed PenHeal, an end-to-end autonomous framework that performs penetration testing and provides remediation recommendations. In addition to identifying vulnerabilities, PenHeal uses LLMs to estimate remediation costs and assess vulnerability risk levels. Meanwhile, Goyal et al. [2025] introduced Pentest Copilot, which improves interaction with tools requiring a graphical interface by using a sandboxed VNC GUI environment. Pentest Copilot also incorporates RAG using Metasploit and MSFVenom as knowledge bases, as well as a file analysis module for inspecting binaries, configuration files, and documents.

More recently, Kong et al. [2025a] proposed VulnBot to address context loss and failed command generation caused by large amounts of generated data and insufficiently explicit actions. VulnBot evaluates LLM performance across several penetration testing phases, including reconnaissance, scanning, and exploitation. It uses a Penetration Testing Graph (PTG) to structure the task and a Check-and-Reflect mechanism to correct actions affected by hallucination. VulnBot can operate in automatic, manual, and semi-automatic modes. However, the study did not provide a detailed analysis of the success rate of each mode across individual penetration testing phases.

Evaluating LLM-based penetration testing requires controlled and reproducible testbeds. Existing studies commonly rely on platforms such as VulnHub [VulnHub [n. d.]], HackTheBox [HACKTHEBOX [n. d.]], published CVEs [CVE [n. d.]], and vulnerable Docker-based environments such as Vulhub [VulHub [n. d.]]. These platforms allow researchers to evaluate whether LLM-based tools can progress through realistic penetration testing tasks, including reconnaissance, exploitation, and privilege escalation. To further standardise benchmarking, Gioacchini et al. [2024] introduced AutoPenBench, which consists of 33 penetration testing tasks packaged in Docker containers. AutoPenBench compares autonomous and assisted agents and shows that assisted agents generally outperform autonomous agents across most phases, except for the discovery phase. However, its tasks are designed as individual containerised challenges rather than complete vulnerable machines such as those found in VulnHub or HackTheBox. The study also found that the success rate of both autonomous and assisted agents declined as the testing progressed into later phases.

2.2 Memorisation in LLM

Memorisation is an important aspect of LLM behaviour, as it can contribute to improved model performance [Hartmann et al. 2023]. Several factors may influence the extent of memorisation, including model size, repeated data in the training dataset, and context length [Carlini et al. 2023]. As a result, evaluating memorisation in LLMs is necessary. One commonly used approach is ground-truth matching, which measures potential memorisation by comparing generated outputs with known reference data. Carlini et al. [2021] used a search engine to verify whether the outputs generated by LLMs appeared identically in publicly available online sources, indicating possible memorisation. Building on this approach, [Nasr et al. 2023] used a large corpus of data rather than search engine results for ground-truth matching.

Potential memorisation has also been identified in code generation. Al-Kaswan et al. [2024] found that LLMs can memorise code logic, documentation, test code, licence information, and dictionaries. Similarly, Chen et al. [2025] demonstrated memorisation in code generation by combining accuracy evaluation with AST-based source code similarity detection tools. Their study also introduced mutated questions to examine whether model performance depended on memorised content. Although performance declined

after mutation, some generated code still passed accuracy tests and retained similarity with the ground truth.

Kong et al. [2025b] identified potential memorisation in LLM-based Automatic Program Repair (APR). Their study compared a dataset that was likely to have been memorised by LLMs with a newly created dataset. They also mutated the code requiring repair and injected memory-triggering details into the prompt to further investigate memorisation in APR. Even after code mutation, the generated fixes still showed average exact-match rates of 68.98% and 51.35%, suggesting that some outputs may have resulted from memorisation.

In the penetration testing domain, Zhu et al. [2025] used CVEs published after the LLMs knowledge cut-off dates to minimise the potential influence of memorisation. They argued that using post-cut-off vulnerabilities helps avoid memorisation and creates a zero-day-like evaluation environment. However, their study focused only on web-specific vulnerabilities rather than complete penetration testing phases.

3 Methodology

3.1 Vulnerable Machines

This study evaluates LLM-based penetration testing on both public and private vulnerable machines. For the public benchmark, four machines were selected from VulnHub: *The Library: 2*, *Sar*, *Symfonos: 2*, and *CengBox: 2*. These machines were selected because prior research has demonstrated that they can be successfully exploited until root access is obtained, making them suitable for controlled benchmarking.

The selected public machines were divided into two difficulty categories. *The Library: 2* and *Sar* were categorised as easy machines, while *Symfonos: 2* and *CengBox: 2* were categorised as medium machines. This selection supports both cross-category comparison and within-category comparison. The easy and medium categories allow the study to compare LLM performance across different difficulty levels, while the inclusion of two machines per category helps capture variation within the same difficulty level.

In addition to the public machines, this study also developed four private vulnerable machines: two easy machines and two medium machines. These private machines were designed to follow the VulnHub difficulty [VulnHub [n. d.]]. The purpose of including private machines is to create unseen test cases that are unlikely to have appeared in LLM training data or public write-ups. Therefore, the comparison between public and private machines allows the study to examine whether LLM-based penetration testing tools perform better on machines that may have been memorised compared with machines that require reasoning in a novel environment.

3.2 Large Language Models

This study uses two LLMs: GPT-4o-2024-05-13 and Llama-3.1-405B-Instruct. These models were selected based on their use in prior research on LLM-based penetration testing [Isozaki et al. 2025]. By using these two models, the study can compare the behaviour of different LLMs under the same testing conditions, including their performance on public and private machines, their response to memory-triggering prompts, and their adaptability under mutation-based scenarios.

3.3 Tools

The main tool used in this study is PentestGPT. PentestGPT was selected because it is one of the established LLM-based penetration testing tools and has been used in previous research to assist penetration testing tasks. In this study, PentestGPT is used as the baseline tool to evaluate how LLMs perform in vulnerable machine exploitation.

A modified version of PentestGPT, referred to as MemPentestGPT, was also developed to investigate the role of memorisation. MemPentestGPT modifies the prompt by including the vulnerable machine name and the source of the machine. Furthermore, MemPentestGPT also use prompt repetition [Leviathan et al. 2025] which repeat the vulnerable machine nameme and its source as shown as in . For private vulnerable machines, the source is labelled as *thesis*. This modification is intended to trigger potential memorisation by providing machine-specific identifiers to the LLM. Further details of the PentestGPT and MemPentestGPT implementation will be explained in the following section.

3.4 Testing the Suggested Commands

During the experiment, each command suggested by the LLM-based penetration testing tool was executed and evaluated in the controlled testing environment. A suggested command was marked as successful if it executed correctly and produced information or access that enabled progression to the next penetration testing step. For example, a command could be considered successful if it discovered a new service, identified a useful file, obtained valid credentials, achieved a reverse shell, or supported privilege escalation.

If a suggested command failed, the testing process continued by requesting or following the next suggested action from the LLM. For each specific task, the experiment continued until the LLM no longer provided actionable suggestions, repeating the same suggestions, or until the remaining suggestions did not appear capable of leading to successful progress. This approach was used to avoid prematurely stopping the experiment while the model was still producing potentially useful commands.

Isozaki et al. [2025] in their research starts a new task after approximately five failed attempts. Following this practice, repeated failure was used as a practical stopping point when the LLM failed to produce useful progress after several attempts. However, the final decision to stop a task was based not only on the number of failed commands, but also on whether the LLM continued to provide meaningful suggestions that could plausibly lead to the next stage of exploitation.

3.5 Mutation Strategies

This study applies mutation-based assessment to examine whether the LLM relies on reasoning or memorisation when generating command suggestions. The mutation strategies are adapted from previous research on memorisation in LLM-based code repair [Kong et al. 2025b]. However, because this study focuses on suggested

penetration testing commands rather than code generation, not all mutation strategies from prior work are applicable.

Kong et al. [2025b] mutation strategies includes variable renaming and variable swapping. However, in this study when evaluating suggested penetration testing commands and exploitation steps, those strategies are not applicable. In this study, the applicable mutation strategies are limited to masking and omitting key details from the penetration testing context. An example of the strategies applied in this study is shown in Figure **??**.

The first strategy is *masking*, where important details in the tool output are replaced with a placeholder such as `<REDACTED>`. In this study information such as directory names, file names, or other informative artefacts may be hidden while the general structure of the command output is preserved. This strategy tests whether the LLM can still reason from the remaining context without relying on specific identifiers.

The second strategy is *omission*, where key details are removed more aggressively from the prompt. Unlike masking, which preserves the structure of the original output, omission provides only partial information to the model. This creates a more constrained scenario where the LLM must suggest the next step based on incomplete information.

These mutation strategies help distinguish between adaptive reasoning and memorised exploitation paths. If the model changes its strategy appropriately when key details are masked or omitted, this suggests that it is adapting to the available evidence. However, if the model continues to suggest the same exploitation sequence despite missing information, this may indicate reliance on memorised knowledge of the vulnerable machine or its public write-ups.

3.6 Ground-Truth Matching for Exact Command Matching

To further evaluate potential memorisation, this study uses ground-truth matching to compare LLM-generated commands with publicly available exploitation write-ups. The purpose of this analysis is to determine whether the commands suggested by the LLM exactly match commands that appear in existing articles or blog posts describing the exploitation process of the selected public machines.

The ground-truth dataset was collected using Google dorking to retrieve public write-ups related to each vulnerable machine. For example, for the *Sar* machine, the following Google query was used:

```
intitle:"Sar" intext:"nmap" -ai
```

This query was used to identify articles that include step-by-step exploitation details. The retrieved articles were then scraped and processed to extract relevant command examples. Regular expressions were then used to compare each command suggested by the LLM against the commands found in the collected write-ups.

A suggested command was marked as an exact command match if it appeared in at least one of the collected articles after normalisation. During this process, machine-specific values were excluded from the comparison, including file names, IP addresses, hostnames, and target system names. However, wordlists were not excluded, as the choice of wordlist may reflect an important part of the exploitation strategy. Therefore, the matching process focused mainly on the command structure, parameters, and the order of those parameters.

This approach allows the study to measure whether the LLM is producing commands that closely resemble public exploitation write-ups. Exact command matching does not prove memorisation by itself, but it provides evidence of potential memorisation, particularly when the same commands are generated for public machines with widely available write-ups. By comparing exact command matches across public and private machines, as well as across baseline and memory-triggering settings, the study can further assess whether memorisation functions as a strength or weakness in LLM-based penetration testing.

4 Results and Discussion

5 Conclusion

6 Limitation

Acknowledgments

To Indonesia Endowment Fund for Education (LPDP) for funding the Master's study and this research.

References

Ali Al-Kaswan, Maliheh Izadi, and Arie van Deursen. 2024. Traces of Memorisation in Large Language Models for Code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering* (Lisbon, Portugal) *(ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 78, 12 pages. doi:10.1145/3597503.3639133

Stella Biderman, USVSN Sai Prashanth, Lintang Sutawika, Hailey Schoelkopf, Quentin Anthony, Shivanshu Purohit, and Edward Raff. 2023. Emergent and predictable memorization in large language models. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NIPS '23)*. Curran Associates Inc., Red Hook, NY, USA. event-place: New Orleans, LA, USA.

Nicholas Carlini, Daphne Ippolito, Matthew Jagielski, Katherine Lee, Florian Tramer, and Chiyuan Zhang. 2023. Quantifying Memorization Across Neural Language Models. arXiv:2202.07646 [cs.LG] https://arxiv.org/abs/2202.07646

Nicholas Carlini, Chang Liu, Ãšlfar Erlingsson, Jernej Kos, and Dawn Song. 2019. The secret sharer: evaluating and testing unintended memorization in neural networks. In *Proceedings of the 28th USENIX Conference on Security Symposium (SEC'19)*. USENIX Association, USA, 267–284. event-place: Santa Clara, CA, USA.

Nicholas Carlini, Florian TramÃ¨r, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ãšlfar Erlingsson, Alina Oprea, and Colin Raffel. 2021. Extracting Training Data from Large Language Models. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, 2633–2650. https://www.usenix.org/conference/usenixsecurity21/presentation/carlini-extracting

Brian Challita and Pierre Parrend. 2025. RedTeamLLM: an Agentic AI framework for offensive security. doi:10.48550/arXiv.2505.06913 arXiv:2505.06913 [cs].

Wentao Chen, Lizhe Zhang, Li Zhong, Letian Peng, Zilong Wang, and Jingbo Shang. 2025. Memorize or Generalize? Evaluating LLM Code Generation with Evolved Questions. doi:10.48550/ARXIV.2503.02296 Version Number: 1.

CVE. [n. d.]. Common Vulnerabilities and Exposures. https://www.cve.org/About/Overview. [Accessed 29-08-2025].

Hanzheng Dai, Yuanliang Li, Jun Yan, and Zhibo Zhang. 2025. RefPentester: A Knowledge-Informed Self-Reflective Penetration Testing Framework Based on Large Language Models. doi:10.48550/arXiv.2505.07089 arXiv:2505.07089 [cs].

Gelei Deng, Yi Liu, Víctor Mayoral-Vilches, Peng Liu, Yuekang Li, Yuan Xu, Tianwei Zhang, Yang Liu, Martin Pinzger, and Stefan Rass. 2024. PENTESTGPT: evaluating and harnessing large language models for automated penetration testing. In *Proceedings of the 33rd USENIX Conference on Security Symposium* (Philadelphia, PA, USA) *(SEC '24)*. USENIX Association, USA, Article 48, 18 pages.

Luca Gioacchini, Marco Mellia, Idilio Drago, Alexander Delsanto, Giuseppe Siracusano, and Roberto Bifulco. 2024. AutoPenBench: Benchmarking Generative Agents for Penetration Testing. doi:10.48550/arXiv.2410.03225 arXiv:2410.03225 [cs].

Dhruva Goyal, Sitaraman Subramanian, Aditya Peela, and Nisha P. Shetty. 2025. Hacking, The Lazy Way: LLM Augmented Pentesting. doi:10.48550/arXiv.2409.09493 arXiv:2409.09493 [cs].

HACKTHEBOX. [n. d.]. Hack The Box: The #1 Cybersecurity Performance Center — hackthebox.com. https://www.hackthebox.com/. [Accessed 27-08-2025].

Andreas Happe and JÃ¼rgen Cito. 2023. Getting pwnâ€™d by AI: Penetration Testing with Large Language Models. In *Proceedings of the 31st ACM Joint European Software*

Engineering Conference and Symposium on the Foundations of Software Engineering. ACM, San Francisco CA USA, 2082–2086. doi:10.1145/3611643.3613083

Andreas Happe and JÃ¼rgen Cito. 2025. On the Surprising Efficacy of LLMs for Penetration-Testing. doi:10.48550/arXiv.2507.00829 arXiv:2507.00829 [cs].

Valentin Hartmann, Anshuman Suri, Vincent Bindschaedler, David Evans, Shruti Tople, and Robert West. 2023. SoK: Memorization in General-Purpose Large Language Models. doi:10.48550/ARXIV.2310.18362 Version Number: 1.

Eric Hilario, Sami Azam, Jawahar Sundaram, Khwaja ImranÂ Mohammed, and Bharanidharan Shanmugam. 2024. Generative AI for pentesting: the good, the bad, the ugly. *International Journal of Information Security* 23, 3 (June 2024), 2075–2097. doi:10.1007/s10207-024-00835-x

Junjie Huang and Quanyan Zhu. 2023. PenHeal: A Two-Stage LLM Framework for Automated Pentesting and Optimal Remediation. In *Proceedings of the Workshop on Autonomous Cybersecurity*. ACM, Salt Lake City UT USA, 11–22. doi:10.1145/3689933.3690831

Isamu Isozaki, Manil Shrestha, Rick Console, and Edward Kim. 2025. Towards Automated Penetration Testing: Introducing LLM Benchmark, Analysis, and Improvements. In *Adjunct Proceedings of the 33rd ACM Conference on User Modeling, Adaptation and Personalization*. ACM, New York City USA, 404–419. doi:10.1145/3708319.3733804

He Kong, Die Hu, Jingguo Ge, Liangxiong Li, Tong Li, and Bingzhen Wu. 2025a. VulnBot: Autonomous Penetration Testing for A Multi-Agent Collaborative Framework. doi:10.48550/arXiv.2501.13411 arXiv:2501.13411 [cs].

Jiaolong Kong, Xiaofei Xie, and Shangqing Liu. 2025b. Demystifying Memorization in LLM-Based Program Repair via a General Hypothesis Testing Framework. *Proceedings of the ACM on Software Engineering* 2, FSE (June 2025), 2712–2734. doi:10.1145/3729390

Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2025. Prompt Repetition Improves Non-Reasoning LLMs. arXiv:2512.14982 [cs.LG] https://arxiv.org/abs/2512.14982

Milad Nasr, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher A. Choquette-Choo, Eric Wallace, Florian TramÂ¨r, and Katherine Lee. 2023. Scalable Extraction of Training Data from (Production) Language Models. doi:10.48550/arXiv.2311.17035 arXiv:2311.17035 [cs].

VulHub. [n. d.]. Vulhub - Open-Source Vulnerable Docker Environments — vulhub.org. https://vulhub.org/. [Accessed 09-09-2025].

VulnHub. [n. d.]. Difficulty ~ VulnHub. https://www.vulnhub.com/difficulty/. Accessed: 2026-05-15.

VulnHub. [n. d.]. Vulnerable By Design VulnHub — vulnhub.com. https://www.vulnhub.com/. [Accessed 27-08-2025].

Jiacen Xu, Jack W. Stokes, Geoff McDonald, Xuesong Bai, David Marshall, Siyue Wang, Adith Swaminathan, and Zhou Li. 2024. AutoAttacker: A Large Language Model Guided System to Implement Automatic Cyber-attacks. doi:10.48550/arXiv.2403.01038 arXiv:2403.01038 [cs].

Yuxuan Zhu, Antony Kellermann, Akul Gupta, Philip Li, Richard Fang, Rohan Bindu, and Daniel Kang. 2025. Teams of LLM Agents can Exploit Zero-Day Vulnerabilities. doi:10.48550/arXiv.2406.01637 arXiv:2406.01637 [cs].

A Research Methods